Assembler language

Reserving and initializing storage

Part 2: character, hexadecimal, and bit strings

Defining characters

- To define character strings, we use **DC** with type **C**
- For example, to define two fields with the characters ”ABC” and “123”:

```
        DC      C'ABC'
        DC      C'123'
```

- The Assembler listing for the above is next

Defining characters

- The Assembler generates the EBCDIC representation of the specified characters:

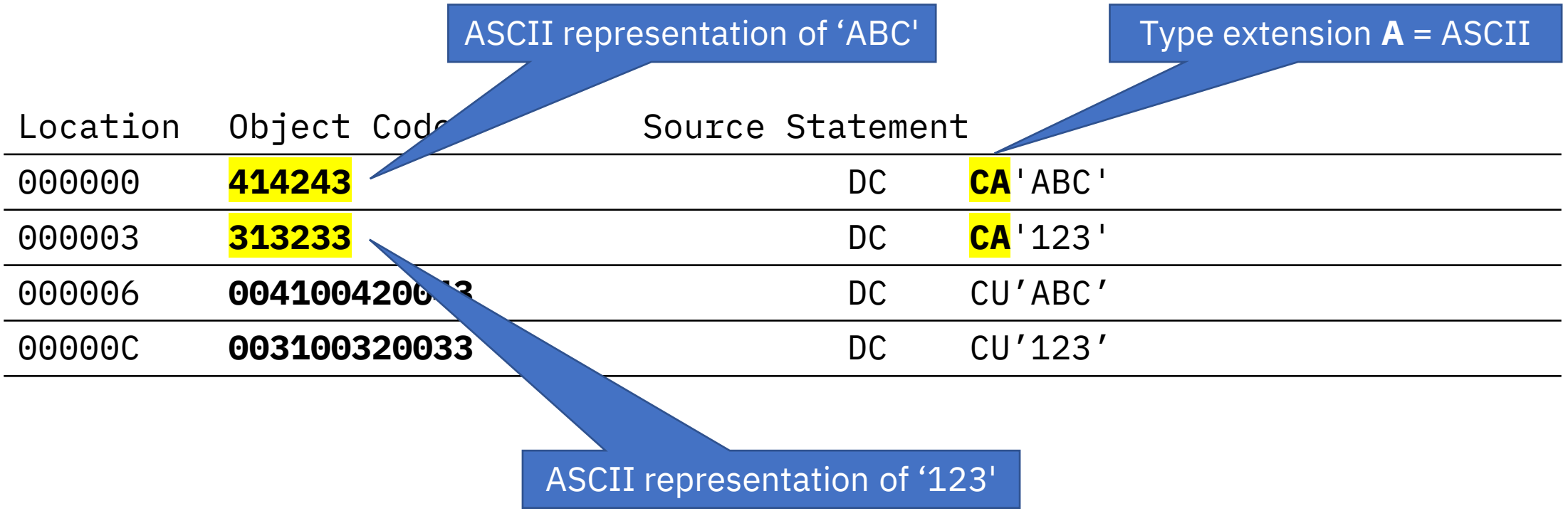
```
Location   Object Code          Source Statement
000000     C1C2C3                    DC     C'ABC'
000003     F1F2F3                    DC     C'123'
```

EBCDIC representation of ‘ABC'

EBCDIC representation of ‘123'

A pause to mention ASCII and Unicode

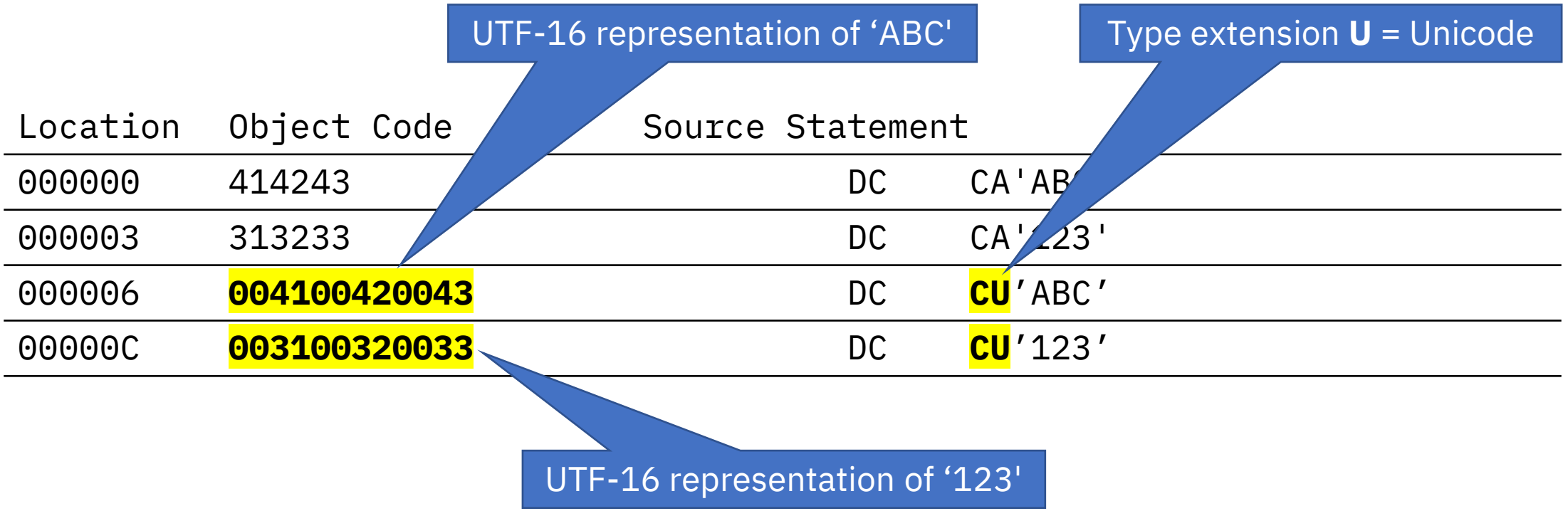
- We won’t cover ASCII or Unicode in the course
- Just for completeness: it is possible to define ASCII or Unicode (UTF-16) characters using a **type extension**:



Location	Object Code	Source Statement	
000000	**414243**	DC	**CA**'ABC'
000003	**313233**	DC	**CA**'123'
000006	**00410042003**	DC	CU’ABC’
00000C	**003100320033**	DC	CU’123’

A pause to mention ASCII and Unicode

- We won’t cover ASCII or Unicode in the course
- Just for completeness: it is possible to define ASCII or Unicode (UTF-16) characters using a **type extension**:



Location	Object Code	Source Statement	
000000	414243	DC	CA'AB
000003	313233	DC	CA'123'
000006	**004100420043**	DC	**CU**’ABC’
00000C	**003100320033**	DC	**CU**’123’

Defining characters - length

- Character strings have variable length, so it's common to specify a field's length

- If the specified length exceeds that of the value, the field is padded on the right with blanks (EBCDIC X'40’):

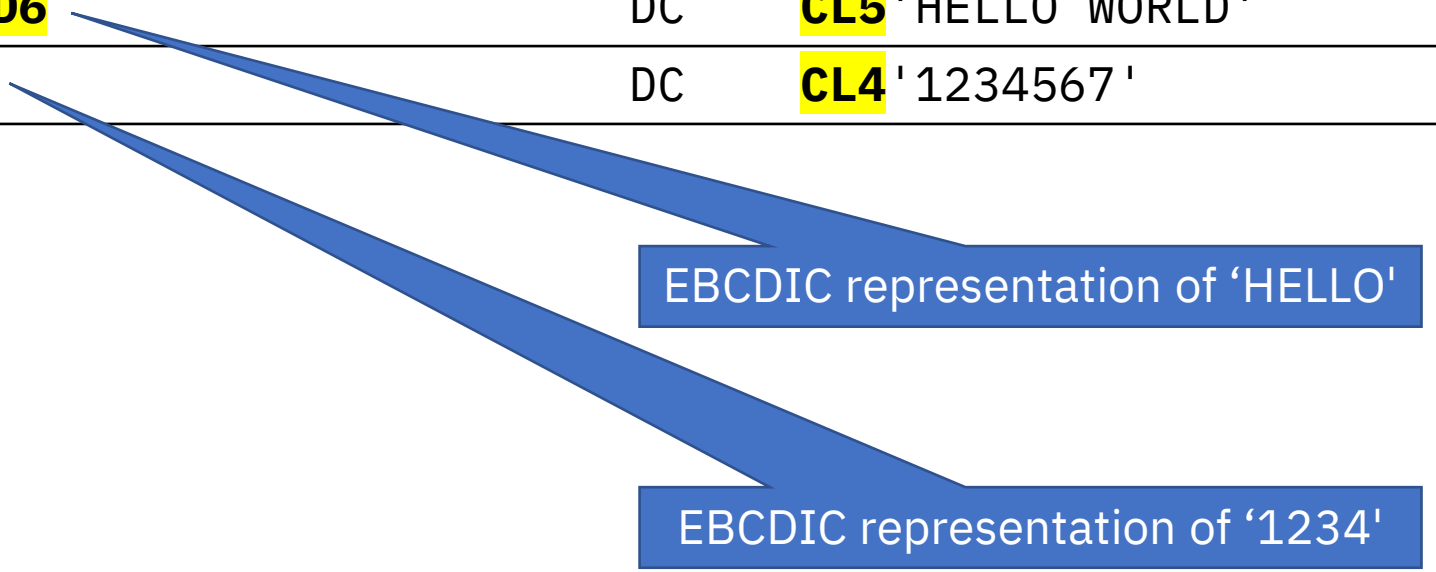
Location	Object Code	Source Statement	
000000	C1C2C3**4040**	DC	**CL5**'ABC'
000005	F1F2F3**40**	DC	**CL4**'123'

- The maximum length of a character field is **256**

Defining characters - truncation

- If the specified length is less than that of the value, the field is truncated on the right:

```
Location   Object Code          Source Statement
000000     C8C5D3D3D6                DC     CL5'HELLO WORLD'
000005     F1F2F3F4                  DC     CL4'1234567'
```



Defining characters - duplication factor

- Question: what is the difference between these two definitions?

Location	Object Code	Source Statement		
000000	**5C5C5C5C5C**	Field1	DC	**5C'*'**
000005	**5C5C5C5C5C**	Field2	DC	**C'*****'**

Defining characters - duplication factor

- Question: what is the difference between these two definitions?

Location	Object Code	Source Statement		
000000	**5C5C5C5C5C**	Field1	DC	**5C'*'**
000005	**5C5C5C5C5C**	Field2	DC	**C'*****'**

- Answer: the length attribute of `Field1` is **1**, the length attribute of `Field2` is **5**
  - Recall that, when using duplication factor, the label refers to the first element
- This can affect how instructions work
  - Example next

Defining characters - duplication factor

- Given these definitions and instruction:

```
Field1      DC      5C'*'
Source      DC      C'ABCDE'
            …
            MVC     Field1,Source       Copy Source to Field1
```

- …after executing the MVC, **`Field1`** will contain **`C'A****'`**
  - The Assembler generates an MVC that copies *1 byte* (it uses the first operand's length for the machine instruction)

Defining characters - duplication factor

- For `Field2`:

```
Field2      DC      C'*****'
Source      DC      C'ABCDE'
            …
            MVC     Field2,Source       Copy Source to Field2
```

- …after executing the MVC, **`Field2`** will contain **`C'ABCDE'`**
  - The Assembler generates an MVC that copies *5 bytes* (the length of `Field2`)

Defining characters - duplication factor

- A duplication factor of zero is useful for defining data structures
- This is an example of a very simple record (details in the next slide):

```
Employee DC    0CL34                Employee:
Emp_name DC    CL20'Joan Smith'      Name
Emp_num  DC    CL6'007777'           Number
Emp_DOB  DC    0CL8                  Date of birth:
DOB_yyyy DC    C'2001'                Year
DOB_mm   DC    C'11'                  Month
DOB_dd   DC    C'25'                  Day
```

Defining characters - duplication factor

Location	Object Code	Source Statement			
000000		**Employee**	**DC**	**0CL34**	Employee:
000000	D196819540E29489	Emp_name	DC	CL20'Joan Smith'	Name
000014	F0F0F7F7F7F7	Emp_num	DC	CL6'007777'	Number
00001A		Emp_DOB	DC	0CL8	Date of birth:
00001A	F2F0F0F1	DOB_yyyy	DC	C'2001'	Year
00001E	F1F1	DOB_mm	DC	C'11'	Month
000020	F2F5	DOB_dd	DC	C'25'	Day

- **Employee** defines a 34-byte field at location 000000
- The duplication factor is zero, so no storage is used
- The fields that follow overlap with **Employee**

Defining characters - duplication factor

Location	Object Code	Source Statement			
000000		Employee	DC	0CL34	Employee:
000000	**D196819540E29489**	**Emp_name**	**DC**	**CL20'Joan Smith'**	Name
000014	F0F0F7F7F7F7	Emp_num	DC	CL6'007777'	Number
00001A		Emp_DOB	DC	0CL8	Date of birth:
00001A	F2F0F0F1	DOB_yyyy	DC	C'2001'	Year
00001E	F1F1	DOB_mm	DC	C'11'	Month
000020	F2F5	DOB_dd	DC	C'25'	Day

- **Emp_name** defines a 20-byte field at location 000000
- The Assembler generates the value X'D196…' (EBCDIC 'Joan Smith')

Defining characters - duplication factor

Location	Object Code	Source Statement			
000000		Employee	DC	0CL34	Employee:
000000	D196819540E29489	Emp_name	DC	CL20'Joan Smith'	Name
000014	**F0F0F7F7F7F7**	**Emp_num**	**DC**	**CL6'007777'**	Number
00001A		Emp_DOB	DC	0CL8	Date of birth:
00001A	F2F0F0F1	DOB_yyyy	DC	C'2001'	Year
00001E	F1F1	DOB_mm	DC	C'11'	Month
000020	F2F5	DOB_dd	DC	C'25'	Day

- **Emp_num** defines a 6-byte field at location 000014
- The field value is ‘007777’ in EBCDIC

Defining characters - duplication factor

Location	Object Code	Source Statement			
000000		Employee	DC	0CL34	Employee:
000000	D196819540E29489	Emp_name	DC	CL20'Joan Smith'	Name
000014	F0F0F7F7F7F7	Emp_num	DC	CL6'007777'	Number
00001A		**Emp_DOB**	**DC**	**0CL8**	Date of birth:
00001A	F2F0F0F1	DOB_yyyy	DC	C'2001'	Year
00001E	F1F1	DOB_mm	DC	C'11'	Month
000020	F2F5	DOB_dd	DC	C'25'	Day

- **Emp_DOB** defines an 8-byte field at location 00001A
- The duplication factor is zero, so no storage is used
- The fields that follow overlap with **Emp_DOB**

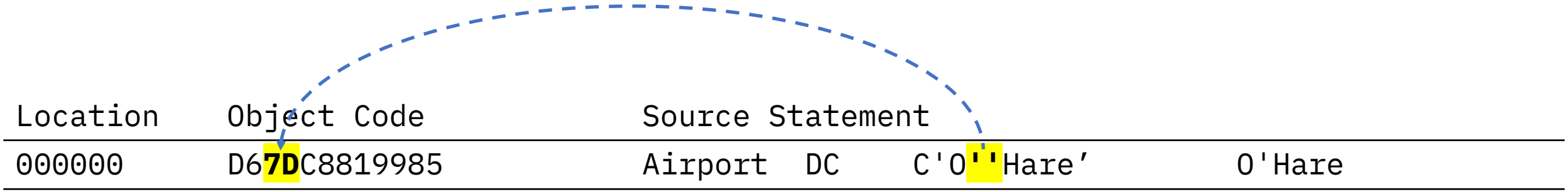
Defining characters - duplication factor

Location	Object Code	Source Statement			
000000		Employee	DC	0CL34	Employee:
000000	D196819540E29489	Emp_name	DC	CL20'Joan Smith'	Name
000014	F0F0F7F7F7F7	Emp_num	DC	CL6'007777'	Number
00001A		Emp_DOB	DC	0CL8	Date of birth:
00001A	**F2F0F0F1**	**DOB_yyyy**	**DC**	**C'2001'**	Year
00001E	**F1F1**	**DOB_mm**	**DC**	**C'11'**	Month
000020	**F2F5**	**DOB_dd**	**DC**	**C'25'**	Day

- The next three fields (year, month, day) overlap with **Emp_DOB**

Defining characters - quote

- The field value is enclosed in single quotes
- To define a character string that contains a quote, use two quotes:



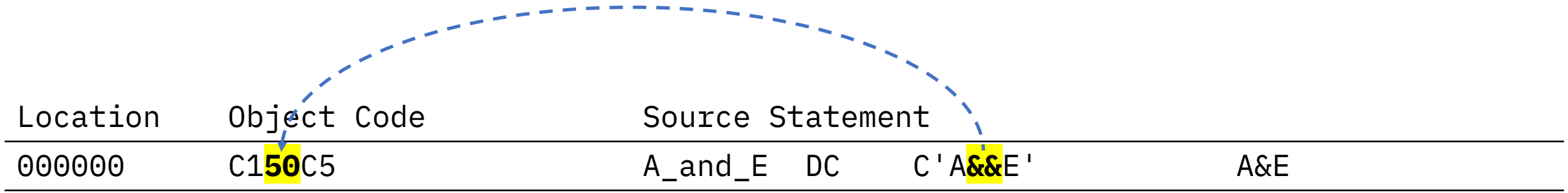
```
Location     Object Code                Source Statement
000000       D67DC8819985               Airport  DC     C'O''Hare'        O'Hare
```

- The two consecutive single quotes result in a single X’7D’ (EBCDIC for quote)

Defining characters - ampersand

- The ampersand character is used to denote variables in the Assembler macro language (not covered in this course)

- To define a character string that contains an ampersand:



```
Location    Object Code              Source Statement
000000      C150C5                   A_and_E  DC     C'A&&E'          A&E
```

- The two consecutive ampersands result in a single X’50’ (EBCDIC for ampersand)

Reserving space

- When using DEFINE STORAGE (DS) for characters, using a value can be useful to specify a length:

Location	Object Code	Source Statement			
000000		Amount1	DS	C'99'	Same as DS CL2
000002		Amount2	DS	C’99.99’	Same as DS CL5
000008		Total	DS	C’9,999.99'	Same as DS CL8

- No data is generated
  - Remember this would cause errors if you intended to use DC but used DS instead!

Defining hexadecimal and bit strings

- To define hexadecimal and bit strings, use the types:
  - **X** for hexadecimal digits
  - **B** for bits

- Hexadecimal and binary types are like characters, with these differences:

  - They are truncated and padded **on the left** with **binary zeroes** (not spaces)

  - Their values can have spaces for readability (like we saw in binary integers)

- Next, a few examples

Examples of hexadecimal strings

Location	Object Code	Source Statement	
000000	F0	DC	X’F0'
000001	**0A**	DC	**X'A'**
000002	00F1	DC	XL2’F1'
000004	A2A3	DC	XL2'A1A2A3'
000006	C1C2C3	DC	X'C1 C2 C3'

- Note that the minimum memory allocation is one byte: an odd number of hex digits results in padding on the left with X’0’

Examples of hexadecimal strings

Location	Object Code	Source Statement	
000000	F0	DC	X’F0'
000001	0A	DC	X'A'
000002	**00F1**	DC	**XL2’F1'**
000004	A2A3	DC	XL2'A1A2A3'
000006	C1C2C3	DC	X'C1 C2 C3'

- When the explicit length exceeds that of the value, the field is padded on the left with hex zeroes
- Note, the length unit is **bytes**

Examples of hexadecimal strings

Location	Object Code	Source Statement	
000000	F0	DC	X’F0'
000001	0A	DC	X'A'
000002	00F1	DC	XL2’F1'
000004	**A2A3**	DC	**XL2'A1A2A3'**
000006	C1C2C3	DC	X'C1 C2 C3'

- When the explicit length is less than that of the value, the field is truncated on the left

Examples of hexadecimal strings

Location	Object Code	Source Statement	
000000	F0	DC	X’F0'
000001	0A	DC	X'A'
000002	00F1	DC	XL2’F1'
000004	A2A3	DC	XL2'A1A2A3'
000006	**C1C2C3**	DC	**X'C1 C2 C3'**

- Using spaces can make the definitions easier to read
- Finally, note that, unlike halfwords or fullwords, hexadecimal strings are not aligned to storage boundaries

Examples of binary strings

Location	Object Code	Source Statement	
000000	F0	DC	B'11110000'
000001	0A	DC	B'1010'
000002	00F1	DC	BL2'11110001'
000004	A2A3	DC	BL2'1010000110100010100011'
000006	C1C2C3	DC	B'1100 0001 1100 0010 1100 0011'

- These statements define the same values as the previous example, but using bit strings instead

Examples of binary strings

Location	Object Code	Source Statement	
000000	F0	DC	B'11110000'
000001	**0A**	DC	**B'1010'**
000002	00F1	DC	BL2'11110001'
000004	A2A3	DC	BL2'1010000110100010100011'
000006	C1C2C3	DC	B'1100 0001 1100 0010 1100 0011'

- When the value has less than 8 bits, it is padded on the left with B’0’

Examples of binary strings

Location	Object Code	Source Statement	
000000	F0	DC	B'11110000'
000001	0A	DC	B'1010'
000002	**00F1**	DC	**BL2'11110001'**
000004	A2A3	DC	BL2'1010000110100010100011'
000006	C1C2C3	DC	B'1100 0001 1100 0010 1100 0011'

- Same as with hex. If the specified length exceeds that of the value, the field if padded on the left with B’0’

Examples of binary strings

Location	Object Code	Source Statement	
000000	F0	DC	B'11110000'
000001	0A	DC	B'1010'
000002	00F1	DC	BL2'11110001'
000004	**A2A3**	DC	**BL2'1010000110100010101000011'**
000006	C1C2C3	DC	B'1100 0001 1100 0010 1100 0011'

- Truncation is on the left

Examples of binary strings

Location	Object Code	Source Statement	
000000	F0	DC	B'11110000'
000001	0A	DC	B'1010'
000002	00F1	DC	BL2'11110001'
000004	A2A3	DC	BL2'1010000110100010101000011'
000006	**C1C2C3**	DC	**B'1100 0001 1100 0010 1100 0011'**

- Spaces are particularly useful to make bit strings more readable
  - Just compare the last row with the one above! Which one do you prefer?

Defining storage

- This completes the sections on reserving and initializing storage using the **DC** and **DS** operations

- There is another way of defining storage: **literals**, our next topic

Assembler language

Reserving and initializing storage

Part 3: literals

Literals

- You can use literals to define data in the instruction itself
- Here is an example:

```
         A       3,=F'100'          Add 100 to register 3
```

- It is the same as:

```
         A       3,FW1              Add 100 to register 3
         …
FW1      DC      F'100'
```

Literals - syntax

- Literals start with an equal sign (“**=**“) followed by the data

- To specify the data, use the same syntax as in **DC**
  - All DC options are valid, ***except duplication factor zero***

- The Assembler places the literals, by default, at the end of the program

Literals - examples

- A few more examples:

	AH	5,=H’20’	Add 20 to register 5
	MVC	Emp_DOB,=C’20011125’	Set employee date of birth
	MVC	Flag1,=X’FFFF’	Move X’FFFF’ to Flag1
	MVC	Flag1,=B’1111 1111 1111 1111’	Same as above, using bits

Literals - benefits

- Literals are convenient because:
  - The data is visible in the instruction in which the literal appears, making the program easier to read
  - You avoid the added effort of defining constants elsewhere
  - Unlike fields defined with **DC** (which can be changed), literals are true constants

Summary

- DC (DEFINE CONSTANT) is used to reserve storage with initial values
- DS (DEFINE STORAGE) is used to reserve storage without initial values
- Literals allow you to specify data directly in the instruction

Summary

- DC (DEFINE CONSTANT) is used to reserve storage with initial values
- DS (DEFINE STORAGE) is used to reserve storage without initial values
- Literals allow you to specify data directly in the instruction

- This concludes the section on defining data
- Next, Assembler instructions we need to have a working program